# PROJECT Report
# TeleGAssistance

Table of Contents

Project Proposal ..... 3
Project Abstract ..... 3
High Level Requirement ..... 3
Conceptual Design ..... 3
Proof of Concept ..... 4
Background ..... 6
Required Resources ..... 6
Projet Design ..... 7
Vision ..... 7
Persona Jack, a primary school teacher ..... 7
Persona Emma, a history teacher ..... **Error! Bookmark not defined.**
Class Diagram ..... 9
Project Progress ..... 12
Week 2 Progress ..... 12

## Project Proposal

### Project Abstract

For this project idea, we wanted to create a Locally hosted AI Assistant that you can speak to through telegram. This is because throughout college I really struggled to stay organized. we tried using various to-do list apps and calendar apps to track my assignments submissions and various deadlines from personal life to school to my work life. However, the issue still prevailed that it simply was too difficult to port over data from canvas to other software's and my own planning like notion or Todo was simply to tedious to setup and keep updating. Therefore, we want to create a software that runs locally on a laptop which utilizes a Local AI LLM as a personal assistant that we can text through my phone. This assistant will be able to text you updates daily about what work you have left to do, what deadlines are approaching and what my days' work routine should look like to stay onto of my workload from various aspects of my life. The intended users are students who have to balance school, legal work and their internships. As an international student who works part time at an internship, we find having hundreds of different deadlines throughout the semester that we simply cannot track by myself. There are similar bots that have been created with the same goal in mind but none that do exactly what we described above.

### High Level Requirement

TeleGAssistant functions as a **proactive academic chief of staff**. From the user's perspective, the product eliminates the need to manually check multiple dashboards by delivering curated, actionable intelligence directly to their phone.

## What the Product Does:

- **Automated Synthesis:** It automatically pulls assignment deadlines, quiz dates, and course announcements from Canvas.
- **Intelligent Briefing:** Every morning, it pushes a natural-language "Daily Briefing" that summarizes the day's workload and upcoming high-priority targets.
- **Proactive Reminders:** It monitors deadlines in real-time and sends "nudge" notifications (e.g., 24h, 2h, and 30m before a due date).
- **On-Demand Querying:** Users can text the bot to ask for current grades, upcoming tasks, or to reschedule personal reminders.

## How it Works (User Flow):

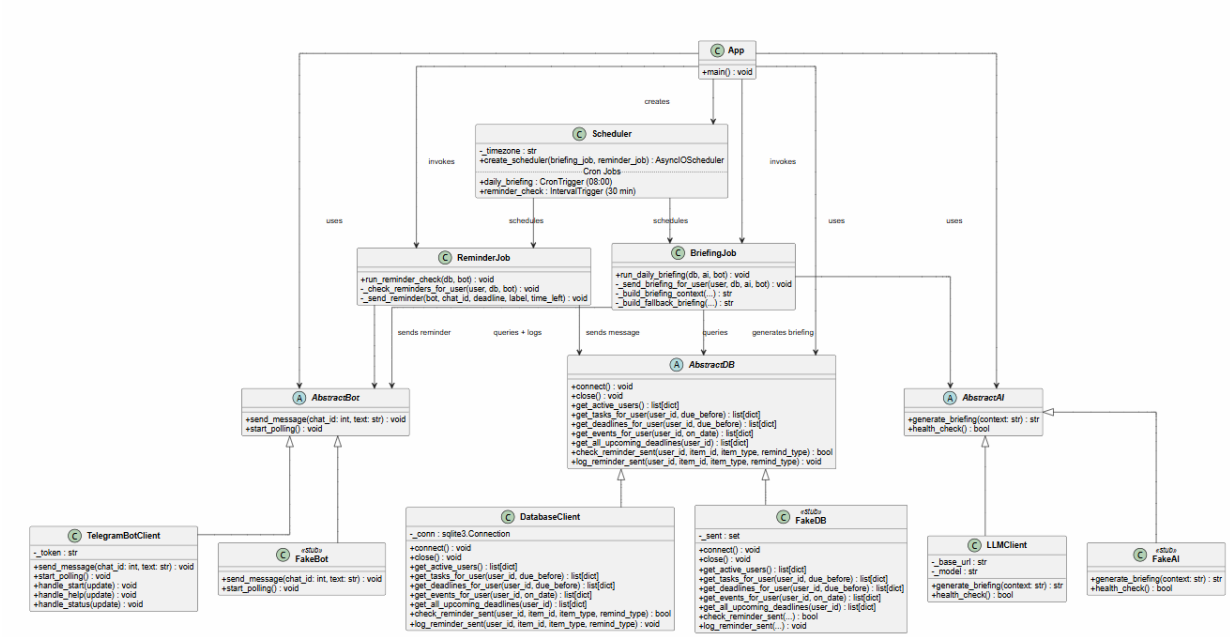
1. **Onboarding:** The user provides their Canvas API token and Telegram Chat ID once.
2. **Passive Monitoring:** The system runs in the background, syncing data every 6 hours without user intervention.
3. **Active Engagement:** The user receives a message at 8:00 AM: *"Good morning! You have a CS exam at 2 PM and a Lab report due by midnight. I recommend starting the report by 4 PM."*
4. **Interaction:** The user can reply, *"Remind me about the report in 2 hours,"* and the bot will update the schedule accordingly.

## Conceptual Design:

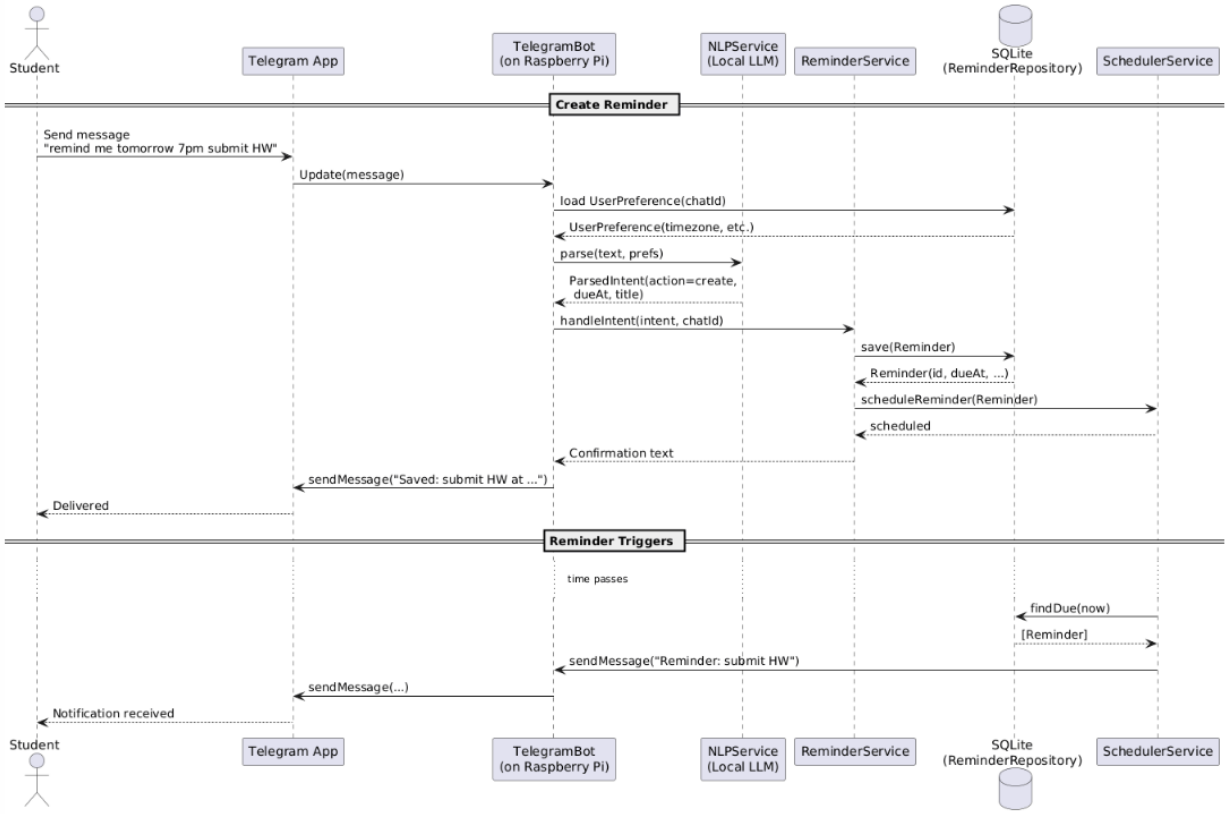
The TeleGAssistant is designed as a locally-hosted, proactive AI middleware that bridges academic data (Canvas) with a conversational interface (Telegram).

- Hardware Architecture: The system is designed to run on consumer-grade local hardware, specifically a Raspberry Pi 5 or a laptop (macOS/Windows), ensuring data privacy by keeping all AI processing "on the edge."
- Software Architecture: We utilize a Modular Monolith pattern. The system consists of four primary layers:
  - Orchestration: Python-based AsyncIO event loop and APScheduler.
  - Intelligence: A local instance of the Gemma 2 2B LLM running via a local inference server.
  - Interface: The Telegram Bot API for asynchronous user interaction.
  - Data: An SQLite database for persistence and a Canvas API integration layer for real-time academic syncing.
- Operating System: cross-platform compatibility via Python.

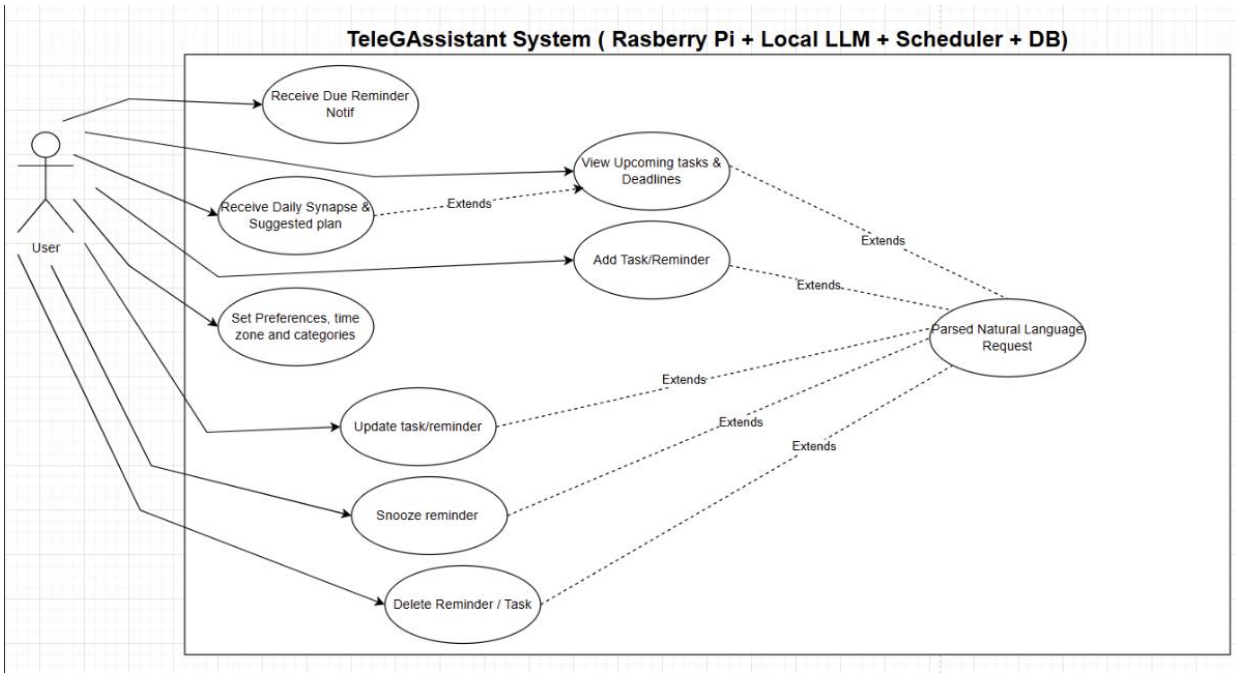
### Class Diagram( as of current instance ):




### Sequence Diagram:




## Use Case diagram:



## Proof of Concept

**http://github.com/varunaditya27/TaskNova%20**

https://github.com/pedrosfaria2/telegram_assistant

**github.com/dome272/Telegram-Reminder-Bot**

**github.com/lazarusking/theaccelbot**

These are some repositories we have found that have existing infrastructure for us to build off of and that use the same or similar coding infrastructure that we plan to use.

## Background

Several different tools that currently exist and are in use offer partial solutions to my described problem. Applications such as Notion and Todo's are known to provide flexible task management and reminders, while other people opt to use Google Calendar as it is known to excel at time-based scheduling. However, it can be argued that these tools can and usually rely heavily on manual configuration and lack the ease of conversational interaction. Most also operate entirely in the cloud, raising privacy concerns and limiting the users' control over their own personal data. Additionally, their AI features, where present, are typically generic productivity assistants rather than tools designed specifically to understand and manage a student's academic workload. There are also several open-source and semi- open-source Telegram bots on GitHub the links of which are attached under my proof-of-concept section that support reminders and task scheduling using the telegram Api system. These examples include simple Telegram reminder bots that allow users to set one-time or recurring notifications using predefined commands such as /start and /list, as well as more advanced bots that apply basic natural language processing to schedule tasks. While these projects demonstrate the feasibility of TeleGAssistant (Our proposed solution), they are generally limited in scope for how much they are able to handle. Most rely on cloud-hosted services, models and tend to use rule-based parsing instead of something more robust such as a local language model and focus on reminders rather than holistic workload planning like I plan my solution to incorporate. I will be using these open-source projects for understanding behind routing the telegram API and how the infrastructure works as well as foundation, but specific source code will not be recycled or reused from any of these projects listed below.

## Required Resources

Hardware:

- Personal Laptop (host machine — no additional hardware required)

Software:

- VSCode (For Development)
- Telegram bot API
- LLM Model for download (https://huggingface.co/google/gemma-2-2b )

## Project Design

## Vision

TeleGAssistant is a locally hosted Telegram bot for students who are juggling academics, internships, and personal deadlines across disconnected tools. It runs on a laptop, uses a local LLM to generate intelligent responses, and gives students a single conversational interface to manage their workload — without cloud dependency or manual setup overhead. Unlike existing tools that are time-based only or require manual input, TeleGAssistant combines conversational AI, scheduling, and holistic workload awareness in one place.

Initial User Feature List:

User can interact with the assistant via Telegram by sending text messages from their phone

User receives a daily briefing covering what's due, approaching deadlines, and a recommended daily work routine

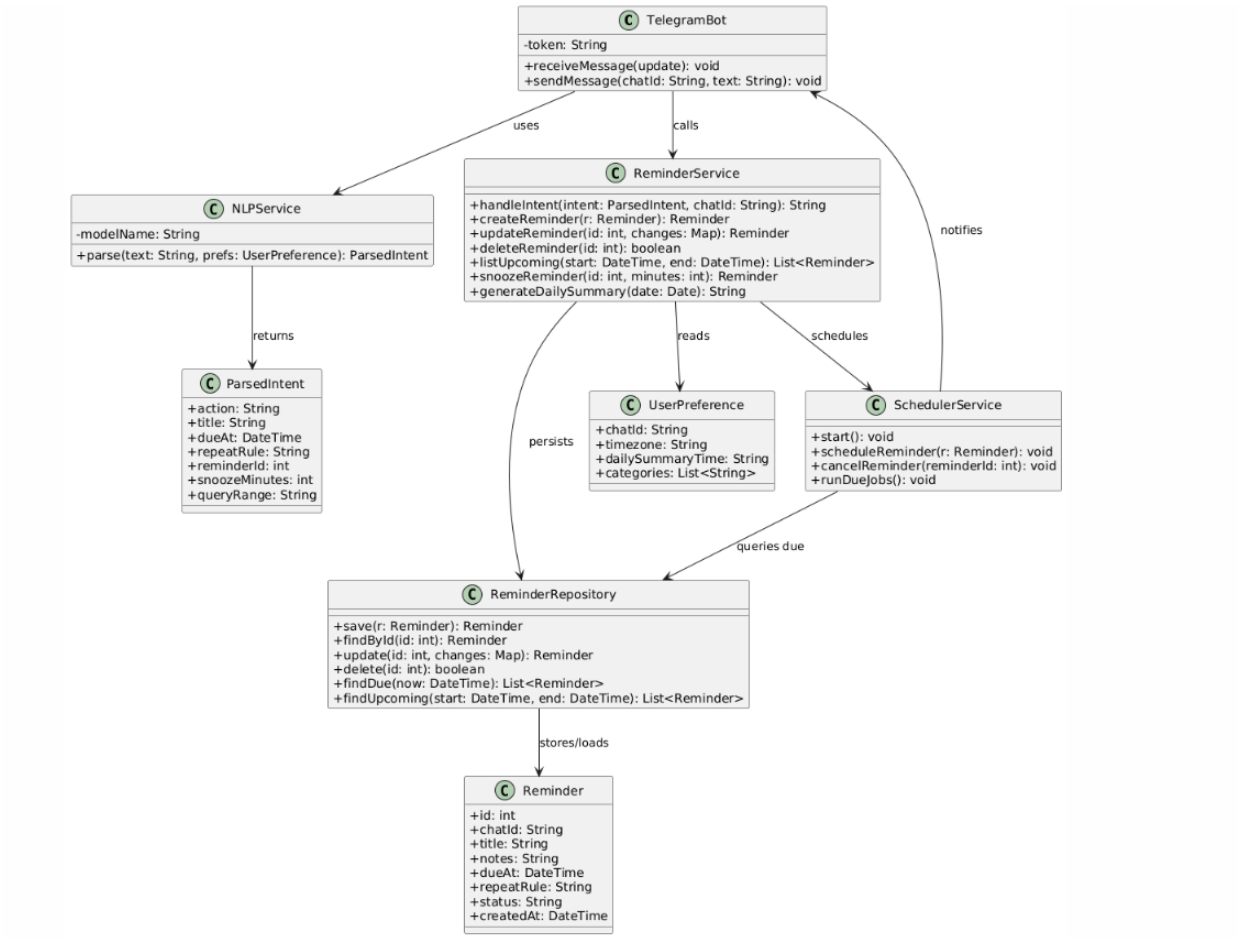
User receives reminders for upcoming deadlines

User can manage scheduling through the bot.

1. **Persona:** Alex Rivera – The overwhelmed student
   - Name: Alex Rivera
   - Age: 21
   - Occupation: Full time computer science student
   - Educational background: currently pursuing degree in comp sci
   - Technology experience;
     - Comfortable with coding programs and tools, social media applications
     - Daily used programs: Canvas, Discord, Telegram
   - Demographic:

    - Lives on campus at a small university
    - Balances Comp sci classes, a part-time job, and social life
  - Why they would use out assistant:
    - Automatic tracking of assignment deadlines
    - Reduce stress produced from managing multiple classes and responsibilities
    - Get daily summaries and reminders without opening multiple different apps.
2. **Persona:** Maya Patel – The organized overachiever
  - Name: Maya Patel
  - Age: 22
  - Occupation: Pre-Med student
  - Educational background: College senior getting bachelor's in Biology
  - Technology experience:
    - Advanced capabilities with digital planners, spreadsheets, and other scheduling tools.
  - Demographic:
    - Lives off-campus with multiple roomates
    - Very disciplined and academically focused
  - Why they would use our assistant:
    - Enhance al already structured lifestyle with AI-powered planning
    - Optimize task prioritization/time management
    - Consolidate scheduling into one system
3. **Persona:** Chris Thompson – The last-Minute Procrastinator
  - Name: Chris Thompson
  - Age: 20
  - Occupation: College student (undeclared major)
  - Educational background: Sophomore exploring career options
  - Technology experience:
    - Competent with basic apps but uncomfortable with complex productivity tools
    - Prefers quick, low-effort solutions
  - Demographic:
    - Lives in a dorm and has a very active social life
    - Is known to delay assignments until right before the deadline
  - Why they would use our assistant:
    - Get automatic reminders before deadlines sneak up
    - Simplify task tracking without manual organization
    - Receive quick prompts that keep them on track.

## Class Diagram




At the very center, we have the App/Main Entry Point. This is the boss that starts everything up. It creates instances of our four main specialized classes:

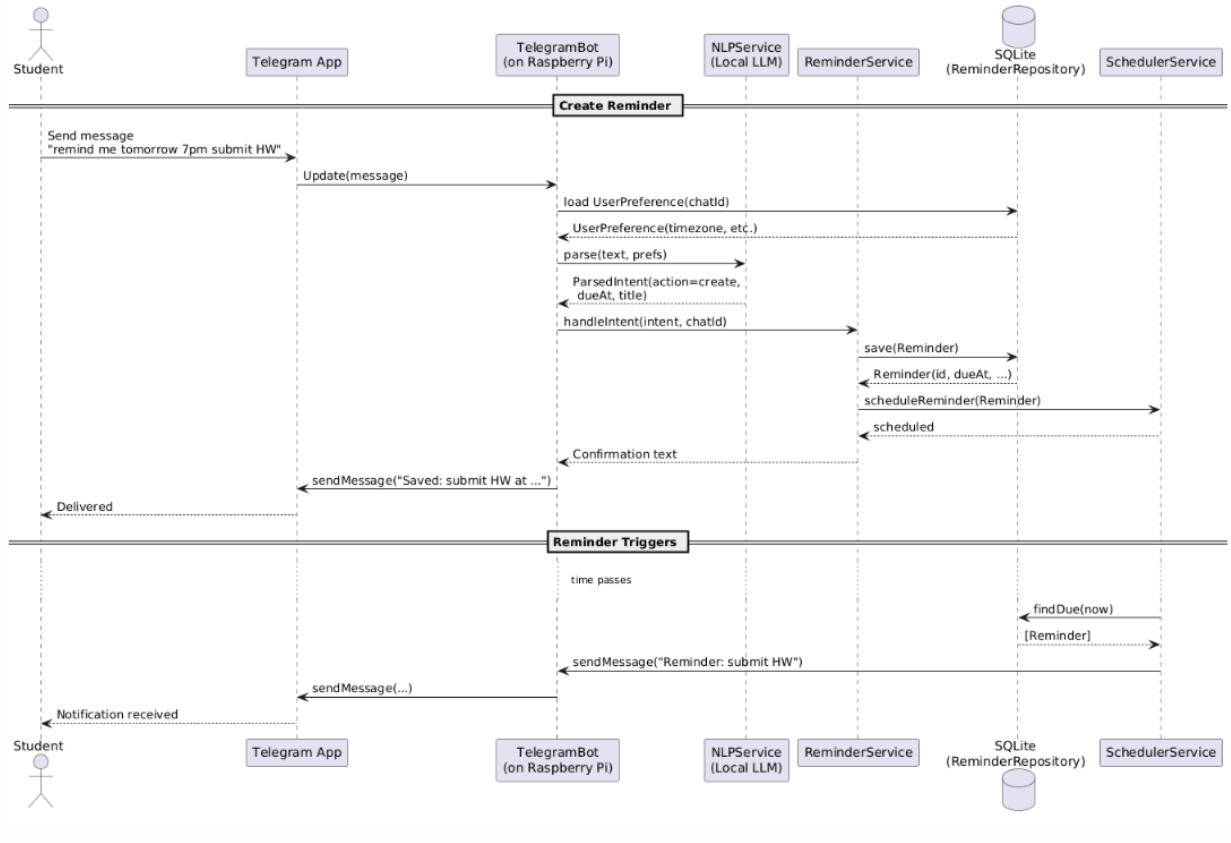
The Database (Data Engineer): This is our long-term memory. It doesn't do anything on its own; it just waits for other classes to ask it to save or retrieve data (like Canvas assignments or user settings).

The TelegramBot (Interface Engineer): This is our 'mouth' and 'ears.' It listens for user texts and sends back the final responses.

The Gemma Interface (AI Engineer): This is the brain. It takes raw text data and turns it into a friendly, human-like paragraph.

The AsyncIOScheduler (Our Automation Layer): This is the 'metronome' of the project. It holds references to the other three. It knows that at 8:00 AM, it needs to grab the Database, pull tasks, hand them to the AI, and then give the result to the TelegramBot.

## Sequence Diagram

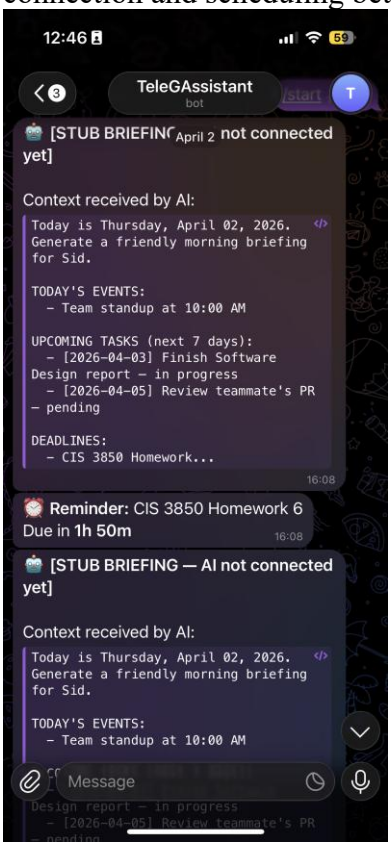



This diagram tracks the "Daily Briefing" lifecycle to show how data moves through the system in real-time.

- Trigger: The Scheduler hits a 08:00 timestamp and wakes up the Briefing Job.
- Fetch: The job queries the Database for today's specific tasks and deadlines.
- Process: The raw data is sent to the AI (Gemma). The AI transforms "Raw Data" (e.g., CS Exam 2pm) into "Natural Language" (e.g., Good morning! Don't forget your exam at 2:00.).
- Deliver: The polished text is passed to the TelegramBot, which pushes the message to the user's phone.
- Log: The system records that the briefing was successful and goes back to sleep.

## Automated Test Results

From the first weeks development the only testing we had the chance to conduct was the connection and scheduling between the telegram bot and our App.py backend as shown below:




As can be seen in the screenshot we successfully managed to connect out telegram bot to our backend locally hosted server to start sending messages to the user based on API keys from the bot and my unique chat id.

# Project Progress

## Week 2 Progress

**Scrum Master:** Sidhant Kumar

**Sprint Goal: Now that we have created the skeleton and stubbed scaffolding its time to start the transition to a running application.**

### Backlog Features

- User receives a real Telegram message from the bot ( not a terminal print )
- User data is persisted in real SQLite Database
- Daily briefing is generated by local LLM model
- Application passes automated unit tests (still need to be developed)

**Bot & Interface Engineer**

| Ticket | Description | Size |
|---|---|---|
| Implement `TelegramBotClient` | Create `bot/telegram_client.py` implementing `AbstractBot`. Wire `send_message()` and `start_polling()` using `python-telegram-bot` so `app.py` can run. | 5 |
| Handle `/start` command | When a user sends `/start`, capture their Telegram chat ID and store them as an active user in the database. Send a confirmation message back. | 3 |
| Implement `/help` and `/status` commands | `/help` returns a list of available commands. `/status` confirms the bot is online. | 2 |

**Data Engineer**

| Ticket | Description | Size |
|---|---|---|
| Design SQLite schema and init script | Define all tables ( `users`, `tasks`, `deadlines`, `events`, `reminders_sent` ) and write `db/init_db.py` to create them. Must match the return shapes in `interfaces.py`. | 3 |
| Implement `DatabaseClient` | Create `db/database_client.py` implementing all 8 `AbstractDB` methods against the real SQLite database. Replaces `FakeDB` in `app.py`. | 5 |
| Write dev seed script | Write `db/seed_dev.py` to populate the database with realistic test data so the team can run the app locally without manually inserting rows. | 2 |

**AI Engineer (Sid)**

| Ticket | Description | Size |
|---|---|---|
| Set up ollama and verify Gemma 2B | Install ollama, pull `gemma2:2b`, and confirm the local inference server responds to a raw HTTP request. | 2 |
| Implement `LLMClient` | Create `ai/llm_client.py` implementing `AbstractAI`. `generate_briefing()` sends context to Gemma and returns a Telegram-ready string. `health_check()` returns `True` / `False` without raising. | 5 |
| Prompt engineering — briefing quality | Iterate on the system prompt and context format until briefing output is concise, accurate, and doesn't hallucinate. Test against at least 3 different context scenarios. | 3 |

**Automation Engineer**

| Ticket | Description | Size |
|---|---|---|
| Write unit tests for `jobs/briefing.py` | Fill `tests/test_briefing.py` using the existing stubs. Cover happy path, LLM failure fallback, Telegram retry logic, and empty data handling. | 3 |
| Write unit tests for `jobs/reminders.py` | Fill `tests/test_reminders.py`. Cover each time window (24h/2h/30min), dedup behavior, and past-due skipping. | 3 |
| End-to-end smoke test — wire real components | Swap stubs for real implementations in `app.py` once each role's work is done. Run startup smoke test and confirm a real briefing hits Telegram. Capture coverage report output. | 3 |

Calculated velocity: 39

# Project Progress

## Week 3 Progress

**Scrum Master:** Sidhant Kumar

**Sprint Goal: With a fully running application, the final sprint focuses on real-world data integrations, presentation readiness, and a clean demo build before the April 22 presentation.**

### Backlog Features

- User receives a daily briefing enriched with live weather data
- User's unread Gmail emails from the last 24 hours are summarized in the briefing
- User can trigger an on-demand briefing at any time via /briefing command
- Canvas assignments are pulled from the live API and persisted as real deadlines
- Application ships with a full automated test suite and coverage report
- All team members have prepared and rehearsed their assigned presentation sections

**Bot & Interface Engineer**

| Ticket | Description | Size |
|---|---|---|
| On-demand `/briefing` command | Let users trigger a briefing manually via command instead of waiting for the scheduled time. | 3 |
| Message formatting pass | Audit all outbound messages for clean Telegram MarkdownV2 rendering. No broken formatting during the demo. | 2 |
| User-facing error messages | If the bot fails or LLM is down, respond with a readable message instead of silent failure. | 2 |
| **PRES-01: Slides — Introduction & Project Overview** | Build slides covering Section 1: group members, project name, problem it solves, competitor products, and how TeleGAssistant differs. Present this section live. | 3 |
| **PRES-02: Personal Reflection** | Prepare and present Section 7 — what you enjoyed, lessons learned technical and non-technical. | 1 |

**Data Engineer**

| Ticket | Description | Size |
|---|---|---|
| Canvas API integration | Pull real assignment deadlines from Canvas into the `deadlines` table on startup. | 5 |
| Scheduled data refresh | Add a cron job to re-sync Canvas data every 24 hours so the DB doesn't go stale. | 3 |
| Data validation + cleanup | Enforce no duplicate deadlines on re-sync, handle null fields, confirm date types are consistent. | 2 |
| **PRES-03: Slides — Project Responsibilities & Technical Implementation** | Build slides for Sections 3 and 4: who did what, tools used, languages, and architecture. Present both sections live. | 3 |
| **PRES-04: Slides — Challenges & Solutions** | Build slides for Section 5: hardest technical and non-technical challenges, how the team overcame them. Present live. | 2 |
| **PRES-05: Personal Reflection** | Prepare and present Section 7. | 1 |

**AI Engineer (Sid)**

| Ticket | Description | Size |
|---|---|---|
| OpenWeatherMap integration | Call the free OpenWeatherMap API on briefing run. Pull current conditions and today's forecast. No OAuth — API key only. | 2 |
| Gmail integration (single-user) | One-time OAuth setup via local script. On briefing run, fetch unread emails from the last 24 hours — subject, sender, snippet. | 4 |
| Context assembly + prompt retuning | Wire Gmail and Weather into `_build_briefing_context()`. Add a length guard (~1500 token ceiling). Re-tune system prompt for all data sources. | 3 |
| **PRES-06: Personal Reflection** | Prepare and present Section 7. | 1 |

**Automation Engineer**

| Ticket | Description | Size |
|---|---|---|
| Full test suite + coverage report | Run complete suite, hit 80%+ coverage on `jobs/`, save the report for submission. | 3 |
| README and deployment guide | Write setup instructions: install deps, configure `.env`, seed DB, run the bot. A stranger should be able to run it cold. | 2 |
| **PRES-07: Live Demo prep** | Package and run the app as a release build (not from IDE). Prepare the demo flow covering all features end-to-end for Section 2. Present the live demo. | 4 |
| **PRES-08: Slides — Unfinished Features, Future Plans & Scrum/Teamwork** | Build slides for Sections 6 and 8: uncut features with mockups, future improvements, meeting cadence, team experience, and Scrum reflection. Present both sections live. | 3 |
| **PRES-09: Personal Reflection** | Prepare and present Section 7. | 1 |

Calculated velocity: 55